

Journal des Erreurs et Solutions

DOCUMENTATION DE DÉPANNAGE

Ce document recense les erreurs rencontrées lors de l'exécution de l'application et détaille les solutions apportées pour chacune d'elles.

1. Erreur de compilation : "variable uploadsDir is already defined"

SYMPTÔME

Échec de la commande `mvn compile` ou `mvn spring-boot:run` avec l'erreur : `variable uploadsDir is already defined in method run(java.lang.String...)`.

CAUSE

La variable `uploadsDir` a été déclarée deux fois dans la méthode `run()` du fichier `DataInitializer.java` (une fois au début de la méthode et une autre fois à l'intérieur d'un bloc `try`).

SOLUTION

La variable située dans le bloc `try` (ligne 288) a été renommée en `pathUploadsDir` pour éviter le conflit de noms.

2. Erreur de lancement : "Web server failed to start. Port 8080 was already in use"

SYMPTÔME

Échec du lancement de `mvn spring-boot:run` car le port 8080 est déjà occupé.

CAUSE

Le conteneur Docker `incidentflow-backend` tournait déjà en arrière-plan (via docker-compose) et écoutait sur le port 8080 de la machine hôte.

SOLUTION

Le conteneur en conflit a été arrêté avec la commande `docker stop incidentflow-backend`. Cela a libéré le port 8080, permettant de lancer l'application en local.

3. Erreur 500 : Impossible d'accéder à la page de gestion des workflows dynamiques

SYMPTÔME

Une erreur 500 "Internal Server Error" (avec le message `Unable to connect to Redis`) est survenue lors de l'appel à l'API `/api/workflows`.

CAUSE

L'application Spring Boot cherchait à se connecter à Redis sur le port par défaut (6379). Cependant, la base Redis lancée via `docker-compose` mappe son port sur `6380` pour la machine hôte.

SOLUTION

Ajout explicite de la configuration de Redis dans `backend/src/main/resources/application.properties` pour pointer vers le port 6380 (avec une variable d'environnement pour être compatible avec Docker) :

```
spring.data.redis.host=${SPRING_DATA_REDIS_HOST:localhost}
spring.data.redis.port=${SPRING_DATA_REDIS_PORT:6380}
```